A. Breadth-First Search (BFS)

BFS uses a FIFO queue (first in, first out). It processes the oldest queued node first.

Step	Processed node	Queue after processing	Path to processed node
1	A	`[B, C]`	A
2	B	`[C, D, E]`	A → B
3	C	`[D, E, F]`	A → C
4	D	`[E, F]`	A → B → D
5	E	`[F, G]`	A → B → E
6	F	`[G]`	A → C → F
7	G	`[]`	A → B → E → G

Final BFS path: `A → B → E → G`

BFS visit order: `A, B, C, D, E, F, G`

Notice that BFS processes `F` before `G`, because `F` was already ahead of `G` in the queue.

B. Depth-First Search (DFS)

DFS uses a LIFO stack (last in, first out). The supplied code pushes successors in reverse order so the first listed successor is processed first.

Step	Processed node	Stack after processing (top at right)	Path to processed node
1	A	`[C, B]`	A
2	B	`[C, E, D]`	A → B
3	D	`[C, E]`	A → B → D
4	E	`[C, G]`	A → B → E
5	G	`[C]`	A → B → E → G

Final DFS path: `A → B → E → G`

DFS visit order: `A, B, D, E, G`

DFS stops immediately when `G` is processed. It does not process the remaining `C` entry on the stack.

Verification against the supplied code

Check	Expected	Executed result
BFS path	A → B → E → G	Match

BFS visit order	A, B, C, D, E, F, G	Match
DFS path	A → B → E → G	Match
DFS visit order	A, B, D, E, G	Match

Both traces match the supplied successor ordering and the manual's expected visit orders.